

Candidate Technical Portfolio

AI Harness and Evaluations Engineer

Shalaka Vinod Jaitapkar

PROFILE	DOMAIN	EVIDENCE	QUALIFICATION
9+ years in quality engineering and UAT	Payments and regulated financial services	Open-source Python AI agent with 78 automated tests	MSc Artificial Intelligence and ISTQB CT-AI

Purpose

This portfolio provides Sharath and the hiring team with a concise, evidence-based view of my suitability for the AI Harness and Evaluations Engineer position. It connects my regulated-domain quality experience with hands-on AI engineering, machine-learning evaluation and human-governed test design.

Core proposition

I bring a combination that is directly relevant to this product organisation: practical payments and quality-governance experience, the ability to translate professional judgment into testable rules, and demonstrated Python implementation of deterministic controls around probabilistic AI output.

Evidence included

- TestScope AI UAT Agent and its public GitHub delivery history
- AI Quality Assurance Framework for governed evaluation across the AI lifecycle
- MSc multimodal recommendation-system research and comparative model evaluation
- AI-enabled test-management architecture using Copilot Studio, Jira and Confluence
- Current extension of TestScope into a human-anchored AI evaluation harness
- Career evidence across UAT, payments, CI/CD, traceability and release governance

github.com/jshalaka/testscope-ai-uat-agent

1 Executive Business Case

The role requires an engineer who can build trustworthy evaluation evidence for AI products in regulated operations. My experience has consistently centred on the same underlying responsibility: converting business requirements and operational risks into test evidence, deciding whether quality criteria are satisfied, and keeping human stakeholders accountable for release decisions.

Why my background is relevant

Business need	Relevant evidence	Value to the product line
Correctness grounded in domain judgment	Payments testing, UAT leadership and business-rule analysis	I can work productively with payments experts and challenge incomplete or unsafe definitions of correct behaviour.
Reliable evaluation controls	Python guardrails, schemas, coverage and traceability in TestScope	I have implemented controls that reject or correct invalid AI output rather than trusting generation alone.
Regression prevention	78-test PyTest suite and incremental GitHub pull requests	I understand how permanent regression evidence develops as the system changes.
Human-governed delivery	Entry/exit criteria, triage, go-live readiness and sign-off	I naturally preserve human ownership for consequential decisions.
Evidence for non-engineers	Test strategies, UAT reporting and leadership-facing solution proposals	I can communicate technical quality and residual risk to business and governance stakeholders.

What differentiates me

My value is not based on claiming a tool-for-tool match with the complete reference stack. It is the intersection of regulated payments knowledge, quality-engineering judgment, AI education and verified hands-on project delivery. The reference stack can be learned; the ability to identify operational risk, establish traceability and defend a release decision has been developed throughout my career.

How I would contribute

- Partner with domain experts to turn professional judgment into reviewable golden cases and clear rubrics.
- Apply risk-based cohorting so critical payment and authentication behaviours receive stricter gates.
- Build repeatable evaluation and regression evidence with Python and CI/CD.
- Mine failures into permanent test cases and communicate limitations without overstating certainty.

2 TestScope AI UAT Agent

TestScope AI is a working local proof of concept that converts software requirements into structured, risk-aware and traceable UAT TestPacks. I built it to explore how generative AI and deterministic quality controls can work together while keeping the tester responsible for final decisions.

Architecture and technology

Layer	Implementation	Quality purpose
User experience	Streamlit	Structured requirement entry, review and controlled export
Model runtime	Ollama with Qwen3 4B	Local generation without sending requirements to a paid external API
Contracts	Pydantic	Typed input and output structures before workflow progression
Guardrails	Python validators	Deterministic rejection of invalid, incomplete or untraceable output
Persistence	SQLite	Execution status, duration, failures and completed TestPack history
Evidence outputs	JSON and OpenPyXL	Machine-readable output and four-sheet business review workbook
Regression	PyTest	78 automated tests with no warnings documented in the repository

Implemented evaluation controls

- Identifier prefixes, uniqueness and duplicate detection
- Unknown acceptance-criterion, risk, ambiguity and assumption references
- Missing steps, invalid structure and incomplete TestPack blocking
- Acceptance-criterion coverage and high-risk criterion coverage
- Deterministic semantic enrichment of test-to-finding relationships
- Removal of unsupported AI-generated relationships
- One controlled correction attempt followed by revalidation
- Human-review status when unresolved information affects a test

Design principle

AI output is untrusted until it passes deterministic validation. This principle directly supports the target organisation's model: AI agents can execute implementation loops, but humans retain ownership of specifications, gates and consequential decisions.

3 Verified Delivery Evidence

The public repository records incremental delivery through eight merged pull requests. The automated suite grew as new capabilities and failure modes were introduced, providing a transparent engineering history rather than a single final demonstration.

Increment	Capability added	Verified regression evidence
PR 1	Local Ollama requirement-analysis prototype, schemas and guardrails	36 tests passed
PR 2	Structured UAT generation, coverage and test-case guardrails	49 tests passed; 100% traceability across seven acceptance criteria
PR 3	Semantic traceability and controlled correction workflow	59 tests passed
PR 4	Streamlit interface and JSON/Excel TestPack export	67 tests passed; workbook inspected
PR 5	Persistent SQLite execution history	74 tests passed
PR 6	Execution status, duration and failure diagnostics	77 tests passed
PR 7	Stronger deterministic traceability and warning removal	78 tests passed; no collection warnings
PR 8	Architecture, screenshots and interview documentation	78 tests passed with no warnings

Failure handling

The workflow follows AI generation, deterministic enrichment, validation, one controlled correction attempt, revalidation and either final TestPack creation or a human-review block. Failed executions retain the failure stage and diagnostic information without being reopened as valid TestPacks.

Limits stated in the project

- Coverage confirms traceability but does not guarantee complete semantic test quality.
- Generated outputs can vary and require human review before use or sign-off.
- The current POC processes one requirement per execution and is not a hosted production service.
- SQLite history is local and the application does not yet provide authentication or role-based access control.

Public evidence

Repository: <https://github.com/jshalaka/testscope-ai-uat-agent>

Merged pull requests: <https://github.com/jshalaka/testscope-ai-uat-agent/pulls?q=is%3Apr%20is%3Amerged>

4 Machine Learning Research

My MSc dissertation developed and evaluated a multimodal Amazon product recommendation system using customer-review text, product metadata, sentiment analysis, machine learning and deep learning. The work provides direct evidence of dataset curation, comparative measurement and transparent model selection.

Dataset and methodology

- Amazon product dataset containing 1,308,489 rows and 20 fields.
- Text preprocessing, feature engineering, sentiment analysis and class-imbalance treatment.
- Comparison of Decision Tree, Random Forest, CatBoost, CNN, LSTM and attention-based approaches.
- Evaluation using accuracy, precision, recall, F1, confusion matrices and ROC analysis.

Selected results

Model	Model family	Reported accuracy	Interpretation
CatBoost	Gradient boosting	92.82%	Best reported classical ML accuracy
CNN	Deep learning	92.72%	Strongest reported deep-learning accuracy
Attention model	Deep learning	85.43%	Useful comparison with higher error levels
LSTM	Deep learning	77%	Lower reported performance than CNN and CatBoost

Relevance to evaluation engineering

This work required selecting metrics, comparing models, interpreting confusion matrices and documenting limitations. For the target role, I am extending this measurement foundation toward cohort design, confidence intervals, judge calibration and production drift analysis.

Academic evidence

MSc Artificial Intelligence, Brunel University London, completed 2024. Dissertation title: Multimodal Amazon Product Recommendation System Using NLP and Machine Learning Techniques.

5 Enterprise AI Test Management Design

I created a leadership proposal for a governed AI-enabled test-management capability using Copilot Studio, Power Automate, Jira and Confluence. The proposal demonstrates how I approach enterprise adoption: begin with a controlled use case, preserve human accountability, establish measurable success gates and scale only after evidence supports the decision.

Operating model

Stage	AI assistance	Human and system control
Plan and analyse	Requirement-quality findings, ambiguity and risk suggestions	Approved strategy, scope and business decisions
Design and approve	First-draft scenarios and structured TestPacks	Lead review and business-owner approval
Execute and defect	Status support and evidence organisation	Named tester results, triage and retest
Report and sign off	Coverage and quality-risk summaries	Exit criteria, residual-risk acceptance and sign-off

Governance controls

- Identity and least-privilege access
- Prompt, rule and template versioning
- Human approval before publication or release use
- Audit history and traceability across Jira and Confluence
- Quality evaluation for completeness, groundedness, duplication and harmful output
- Security, data classification, retention and controlled connector design

Delivery approach

The proposal uses discovery, proof of concept, pilot and scale phases. Measures cover review time, acceptance rate, requirement coverage, unapproved publication, adoption and integration reliability. Targets are intentionally dependent on a confirmed baseline rather than invented improvement figures.

6 AI Quality Assurance Framework

Alongside the TestScope implementation, I designed an end-to-end AI Quality Assurance Framework to define how AI-enabled systems should be evaluated, governed and monitored from specification through production. The framework connects technical evaluation evidence with human accountability and risk-based release decisions.

Framework objectives

- Define measurable quality expectations before selecting metrics or release thresholds.
- Anchor correctness in independently reviewed domain evidence rather than model-generated labels.
- Combine deterministic validation, model-based evaluation and human professional judgment.
- Apply stricter evidence requirements to high-impact payment, authentication and data-handling paths.
- Preserve traceability from specification and evaluation through release, monitoring and remediation.

Lifecycle design

Lifecycle stage	Quality mechanism	Decision evidence
Specification	Intended use, prohibited use, risk classification and human-only actions	Approved evaluation objectives and control boundaries
Data and golden cases	Human-anchored examples, provenance, cohorts and annotation guidance	Reviewed ground truth with ownership and version history
Model and agent evaluation	Deterministic checks, rubrics, LLM judge, adversarial cases and tool-path assessment	Per-case verdicts, error classes and calibrated scores
Pre-release gating	Baseline comparison, cohort thresholds, sample sufficiency and critical-failure rules	Merge and release recommendation with limitations
Production monitoring	Sampled evaluation, telemetry, drift indicators and alert thresholds	Trend evidence and human investigation queue
Failure learning	Root-cause analysis and promotion of confirmed failures into permanent tests	Traceable regression case and verified remediation

Governance and reporting

The framework requires version control for datasets, prompts, judges, rubrics and thresholds; named human approval at specification, merge and release points; documented exceptions and residual risks; and evidence written so engineering, business, model-risk and audit stakeholders can understand what was tested, what passed, what remains uncertain and who approved the decision.

Connection to the target role

This framework directly addresses the role's emphasis on domain-anchored golden datasets, calibrated evaluation gates, CI regression controls, production drift detection, the failure flywheel and evidence suitable for external model-risk review. It is a completed design framework; implementation of its TestScope extension is described separately as active development.

7 Current AI Harness and Evaluations Extension

I am currently extending TestScope from deterministic validation into a broader AI assurance harness aligned to the target role. This work is in progress and is presented as active development, not completed production experience.

Planned evaluation lifecycle

Stage	Implementation objective	Evidence produced
Specification	Define intended use, prohibited use, risks and human-only decisions	Evaluation strategy and risk taxonomy
Golden data	Create payment-domain cases independently of model output	Versioned cases, expected rules and provenance
Evaluation	Combine deterministic checks with rubric-based model evaluation	Per-case scores, error classes and traces
Calibration	Compare LLM-judge verdicts with human labels	Agreement, precision, recall, F1 and Cohen's kappa
Statistical gate	Apply cohort thresholds with honest sample limitations	Baseline comparison and release decision
CI/CD	Run approved suites for relevant changes	Merge-blocking regression report
Production simulation	Sample traffic, auto-score and detect degradation	Drift trends, alerts and human-review queue
Failure flywheel	Promote confirmed failures into permanent cases	Incident-to-regression traceability

Technical learning path

The extension will use Python and PyTest as the core, then add LangGraph and LangSmith for workflow traces and experiments, PostgreSQL/pgvector for evidence and retrieval, GitHub Actions for merge gates, and OpenTelemetry/Grafana concepts for production monitoring. The complete reference stack is treated as a learning roadmap rather than current experience.

Why this matters

The extension joins my existing strengths in UAT, payments, traceability and release governance with the statistical and operational disciplines required to own AI evaluation evidence. It also creates a public artefact that can be challenged, reproduced and improved.

8 Professional and Domain Evidence

Experience area	Relevant responsibility	Connection to the target role
Capgemini UAT leadership	Strategy, scope, entry/exit criteria, business testers, triage and sign-off	Human gates, evidence ownership and release decisions
Cognizant payments	SIT/UAT, API, database and regression testing for payment platforms	Daily collaboration with payment-domain rules and operational risk
Accenture automation	Selenium-Cucumber, Jenkins and enterprise workflow validation	Automation discipline and CI/CD familiarity
TCS BFSI quality	Regression, integration, API, database and performance testing	Production-risk thinking across repeated delivery cycles
Cross-career governance	Jira/Confluence evidence, triage, status and stakeholder reporting	Traceability and communication beyond engineering

Relevant certifications and education

- ISTQB Certified Tester AI Testing v1.0, passed 12 July 2026, certificate 26-CT-AI-279789-12.
- ISTQB Certified Tester Foundation Level.
- MSc Artificial Intelligence, Brunel University London.
- MBA IT Operations, NMIMS Mumbai.
- Master of Computer Applications.

Professional working principles

- Do not treat model output as evidence of its own correctness.
- Use stricter controls for high-impact payment, authentication and data-protection paths.
- Report uncertainty and limitations rather than turning small samples into confident claims.
- Keep specifications, evaluation results, changes and approvals traceable.
- Require humans to own consequential decisions even when agents perform implementation work.

9 Proposed Contribution and Evidence Index

First 90 days

Period	Contribution focus	Expected outcome
Days 1 to 30	Build domain literacy, understand platform patterns, review existing datasets, gates and failure history	Clear risk map, evidence inventory and agreed evaluation priorities
Days 31 to 60	Strengthen cohorts, calibration checks and permanent regression coverage with domain experts	More defensible gates and improved traceability from failures to tests
Days 61 to 90	Improve CI evidence, drift review and risk-facing reporting within team standards	A repeatable evaluation rhythm trusted by engineering and stakeholders

Supporting evidence

Item	Evidence	Status
TestScope AI UAT Agent	Public GitHub repository, README, source structure and merged PR history	Implemented
Automated regression	78 PyTest tests with no warnings documented in README and PR 8	Implemented
MSc research	Final dissertation with dataset, methodology, models and comparative results	Completed
CT-AI	ISTQB Certified Tester AI Testing v1.0 certificate	Completed
AI test-management proposal	12-slide architecture, governance, roadmap and success-measure proposal	Completed design
AI Quality Assurance Framework	End-to-end evaluation, gating, monitoring and governance design	Completed design
Evaluation harness roadmap	Role-mapped architecture and fourteen-day learning/build plan	Active development

Closing statement

I am motivated by the responsibility at the heart of this role: establishing evidence that an AI product behaves correctly enough for regulated operations and being prepared to stop a release when the evidence is insufficient. I offer tested quality judgment, payments-domain experience, AI education and a practical engineering foundation that I am actively deepening through open, reviewable work.